

Introdução ao OpenAPI e Springdoc com Spring Boot

Como documentar e configurar suas APIs de forma automatizada

<https://swagger.io/docs/>

<https://www.openapis.org/>

<https://springdoc.org/>

O que é OpenAPI?

Definição:

- OpenAPI é uma especificação para descrever APIs RESTful de forma padronizada.
- Anteriormente conhecida como Swagger.
- Facilita o desenvolvimento, documentação e teste de APIs, permitindo que humanos e máquinas entendam a estrutura da API.

Estrutura de uma Especificação OpenAPI

Componentes principais:

- Paths: Endpoints da API.
- Operations: Métodos HTTP (GET, POST, etc.).
- Schemas: Modelos de dados usados pela API (ex: JSON).
- Parameters: Parâmetros de consulta, cabeçalhos, corpo, etc.
- Responses: Tipos de respostas possíveis (200, 404, etc.).

Formato:

- Arquivos no formato JSON ou YAML.

O que é Springdoc?

Definição:

- Springdoc é uma biblioteca que gera automaticamente a documentação OpenAPI 3 a partir das suas APIs Spring Boot.
- Substitui o Swagger clássico com integração simplificada no ecossistema Spring Boot.

Por que usar Springdoc?

- Integração simples com Spring Boot.
- Gera a documentação automaticamente baseada em anotações e convenções do Spring.
- Possui uma interface gráfica (Swagger UI) embutida.

Dependências Necessárias

Adicionar no pom e properties:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

```
springdoc.api-docs.enabled=true
springdoc.swagger-ui.enabled=true
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.api-docs.path=/api-docs
```

Acessar:

<http://localhost:8080/{context-path}/swagger-ui/index.html>

Customizações

OpenAPIDefinition:
Configura as informacoes
basicas da api

```
1 usage
2 @OpenAPIDefinition(
3     info = @Info(
4         title = "API Exemplo",
5         version = "1.0",
6         description = "Documentação da API Exemplo",
7         contact = @Contact(name = "Suporte", email = "suporte@exemplo.com")
8     )
9 )
10 @SpringBootApplication
11 public class AvaliadorDeProjetosApplication {
12     public static void main(String[] args) {
13         SpringApplication.run(AvaliadorDeProjetosApplication.class, args);
14     }
15     /*@Bean
16     public CommandLineRunner demo(AlunoRepository repository){
17         return (args) ->{
18             Endereco en = new Endereco("1345", "Medianeira",
19                 "97010340", "11", "Santa Maria", "RS");
20
21             repository.save(new Aluno("José",
22                 "189344", "pietra@gmail", en));
23         };
24     }*/
25 }
```

Customizações

OpenAPIDefinition: Swagger resultado

 **Swagger**
Supported by SMARTBEAR

/avaliador-de-projetos/api-docs

Explore

API Exemplo

1.0 OAS 3.0

/avaliador-de-projetos/api-docs

Documentação da API Exemplo

[Contact Suporte](#)

Servers

http://localhost:8080/avaliador-de-projetos - Generated server url

--

Documentar Métodos e Parâmetros

Cada método e parâmetro será documentado automaticamente na interface Swagger UI.

```
@PostMapping()  
@Operation(summary = "Criar um novo usuário", description = "Cria um novo usuário e o adiciona à lista")  
@ApiResponses(value = {  
    @ApiResponse(responseCode = "201", description = "Usuário criado com sucesso",  
        content = @Content(mediaType = "application/json",  
            schema = @Schema(implementation = Aluno.class))),  
    @ApiResponse(responseCode = "400", description = "Dados inválidos fornecidos", content = @Content)  
})  
public void salvar(@RequestBody Aluno aluno) { this.service.salvar(aluno); }
```

Documentar Métodos e Parâmetros

Cada método e parâmetro será documentado automaticamente na interface Swagger UI.

GET `/aluno/{id}` Buscar usuário por ID

Retorna um usuário correspondente ao ID fornecido

Parameters [Try it out](#)

Name	Description
id * required integer(\$int64) (path)	ID do usuário a ser buscado

Responses

Code	Description	Links
200	Usuário encontrado Media type application/json Controls Accept header. Example Value Schema	No links
404	Usuário não encontrado	No links

```
{
  "id": 0,
  "uuid": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "nome": "string",
  "matricula": "string",
  "email": "string",
  "endereco": {
    "complemento": "string",
    "bairro": "string",
    "cep": "string",
    "numero": "string",
    "cidade": "string",
    "uf": "string"
  }
}
```

Explicação das Anotações

- `@Operation`: Anota a descrição de um endpoint. Use `summary` para um resumo e `description` para mais detalhes sobre o que o método faz.
- `@ApiResponse`: Define as respostas possíveis que o endpoint pode retornar, como status HTTP 200, 404, 400, etc., junto com uma descrição do que significa cada código de resposta.
- `@Parameter`: Utilizado para documentar os parâmetros do método, seja `@PathVariable`, `@RequestParam`, ou `@RequestBody`.
- `@Content` e `@Schema`: São usados dentro de `@ApiResponse` para descrever o tipo de retorno do endpoint e o formato do objeto de resposta.

Documentar o RestController

- Para documentar um RestController, você deve usar anotações como, @Tag para agrupar controladores

```
@RestController
@RequestMapping("/aluno")
@Tag(name = "Alunos", description = "Path relacionado a operações de alunos")
public class AlunoController {
    // ...
}
```

Servers

http://localhost:8080/avaliador-de-projetos - Generated server url

Alunos Path relacionado a operações de alunos



Documentar o RestController

- Para documentar uma Entidade, usar anotações como @Schema.

```
@RequiredArgsConstructor
@Schema(description = "Entidade que representa um aluno no sistema")
public class Aluno {

    @Id @GeneratedValue(
        strategy =
            GenerationType.IDENTITY)
    @Schema(description = "ID do usuário", example = "1")
    private Long id;
    @UuidGenerator
    private UUID uuid;
    @NotNull
    @Schema(description = "Nome do usuário", example = "Felipe")
    private String nome;
    @NotNull
    private String matricula;
    @NotNull
    private String email;
```

Schemas

```
Aluno ▾ {
  description:      Entidade que representa um aluno no sistema
  id                > [...]
  uuid              > [...]
  nome              ▾ string
                   example: Felipe
                   Nome do usuário
  matricula         > [...]
  email             > [...]
  endereco          Endereco > {...}
}
```